

Extended Kalman Filter

A Deep Mathematical and Practical Guide for Multi-Sensor Robot Localisation

Notation: Thrun, Burgard & Fox — Probabilistic Robotics

Multi-Sensor Fusion Project

Husarion UGV — Local/Global EKF with Wheel, IMU, LiDAR, GPS

May 9, 2026

Abstract

Complete self-contained guide to the Extended Kalman Filter for multi-sensor robot localisation. Notation throughout follows Thrun, Burgard & Fox, *Probabilistic Robotics* (MIT Press, 2005). State mean μ , covariance Σ . Linear: A_t, B_t, C_t . Nonlinear models $g(u_t, \mu_{t-1})$ and $h(\mu_t)$ with Jacobians G_t, H_t . Process noise covariance R_t , measurement noise covariance Q_t . Topics: (i) multivariate Gaussian foundations; (ii) Bayes filter; (iii) linear KF derivation; (iv) EKF and Jacobian linearisation; (v) full `robot_localization` dual-EKF architecture — local node, global node, `navsat_transform_node`, message flow, frame hierarchy; (vi) deep theory of each odometry source — wheel, IMU, visual, LiDAR — including physics, kinematics, and noise models; (vii) tuning and failure modes.

Contents

1	Notation Convention	2
2	Introduction and Motivation	2
3	Mathematical Foundations	3
3.1	Random Variables and Probability Density Functions	3
3.2	The Multivariate Gaussian Distribution	3
3.2.1	Properties of the Covariance Matrix	3
3.2.2	The Mahalanobis Distance	3
3.2.3	Why Gaussian?	3
3.2.4	Product of Two Gaussians	4
3.3	Conditional Gaussian	4
4	The Bayes Filter	4
4.1	Belief and the Filtering Problem	4
4.2	Recursive Filtering: Predict and Update	4
4.2.1	Derivation	4
4.2.2	From Bayes to Kalman	5
5	The Linear Kalman Filter	5
5.1	System Model	5
5.2	KF Algorithm	5
5.3	Derivation of Prediction Step	6
5.4	Derivation of Correction Step	6

5.5	The Kalman Gain: Intuition	6
5.6	Joseph Form: Numerical Stability	7
5.7	Optimality	7
6	The Extended Kalman Filter	7
6.1	Why the Linear KF Fails for Robots	7
6.2	First-Order Taylor Expansion	7
6.3	The Jacobian Matrix	8
6.4	Differential-Drive Motion Jacobian	8
6.5	Observation Jacobian Examples	8
6.6	Full EKF Algorithm	9
6.7	Worked Example: Differential-Drive EKF One Step	9
6.8	Practical Issues	10
6.8.1	Angle Wrapping	10
6.8.2	Quaternions for 3D Orientation	10
6.8.3	Uncertainty “Breath”	11
6.8.4	Mahalanobis Gating for Outlier Rejection	11
7	Local and Global Frames: The Dual EKF Architecture	11
7.1	ROS Frame Hierarchy (REP-105)	11
7.2	Why Two Frames? Why Two EKFs?	12
8	robot_localization Architecture	12
8.1	Node Topology	12
8.2	Node-by-Node Walkthrough	13
8.2.1	ekf_node_local (Local EKF)	13
8.2.2	ekf_node_global (Global EKF)	14
8.2.3	navsat_transform_node	15
8.2.4	imu_filter_madgwick (Pre-Filter)	16
8.2.5	LiDAR Odometry Node (e.g., LIO-SAM)	17
8.3	Topic and TF Flow Summary	17
8.4	Why Each Sensor Goes Where	17
8.5	The Covariance Cross-Term Leak (Why GPS is Excluded in this Project)	17
8.5.1	The Mechanism	18
8.5.2	The Leak Path	18
8.5.3	Compounding	18
8.5.4	Numerical Walk-Through (Project Values)	18
8.5.5	Why the Canonical Tutorial Doesn’t Suffer This	19
8.5.6	The Sensor Consensus Solution	19
8.5.7	Alternatives if Global Anchoring is Required	19
9	The EKF Execution Loop	20
9.1	Local EKF Pseudocode	20
9.2	Multi-Sensor Update Order	21
9.3	Prediction-Only Periods	21
10	Wheel Odometry: Theory and Practice	21
10.1	Hardware: Quadrature Encoders	21
10.2	Differential-Drive Kinematics	21
10.3	Pose Integration	22
10.4	What the EKF Receives	22
10.5	Error Sources	22

10.6 Noise Model	23
11 IMU: Theory and Practice	23
11.1 MEMS Inertial Sensors	23
11.1.1 Accelerometer	23
11.1.2 Gyroscope (Coriolis MEMS)	24
11.1.3 Magnetometer (9-DOF IMUs)	24
11.2 Allan Variance Noise Model	24
11.3 Orientation Integration	25
11.4 Attitude from Accelerometer (Static Case)	25
11.5 Madgwick Complementary Filter	25
11.6 What the EKF Receives	25
12 Visual Odometry: Theory and Practice	26
12.1 What VO Computes	26
12.2 Camera Model	26
12.3 Feature-Based VO Pipeline	26
12.3.1 Step 1: Feature Detection	26
12.3.2 Step 2: Feature Description	26
12.3.3 Step 3: Feature Matching	26
12.3.4 Step 4: RANSAC for Outlier Rejection	27
12.3.5 Step 5: Motion Recovery	27
12.4 Direct VO Methods	27
12.5 Visual-Inertial Odometry (VIO)	28
12.6 VO Output for the EKF	28
12.7 Failure Modes	28
13 LiDAR Odometry: Theory and Practice	28
13.1 Time-of-Flight Range Measurement	28
13.2 Iterative Closest Point (ICP)	29
13.2.1 Algorithm	29
13.2.2 Why SVD Yields Optimal Rotation	29
13.3 Point-to-Plane ICP	29
13.4 Normal Distributions Transform (NDT)	30
13.5 LOAM and LIO-SAM	30
13.6 LiDAR Output for the EKF	30
13.7 Failure Modes	30
14 GPS / GNSS	31
14.1 Principle	31
14.2 Accuracy Tiers	31
14.3 What GPS Provides	31
14.4 GPS in the Filter	31
15 Tuning the Filter	31
15.1 Process Noise R_t	31
15.2 Measurement Noise Q_t	32
15.3 Initial Covariance Σ_0	32
15.4 Validation: Innovation Whiteness	32
15.5 Common Failure Modes	33
16 Beyond the EKF	33

16.1 Unscented Kalman Filter (UKF)	33
16.2 Particle Filter	33
16.3 Factor Graphs and iSAM	33
17 Putting It All Together	33
17.1 End-to-End Data Flow (this Project)	33
17.2 Why Each Component Exists	34
17.3 Comparison: Canonical vs Project Pattern	34
18 Summary Cheat Sheet	35
18.1 The KF in One Page	35
18.2 The EKF in One Page	35
18.3 Sensor Quick Reference	36
18.4 Frame Hierarchy Quick Reference	36
18.5 Topic / Node Quick Reference	36
18.6 Notation Cheat Sheet	36
Closing Notes	36

1 Notation Convention

This document follows Thrun-Burgard-Fox *Probabilistic Robotics* convention exclusively. Reader from engineering KF literature: be aware of the symbol swap.

Quantity	Symbol (this doc)	Engineering literature
State mean	μ_t	$\hat{\mathbf{x}}_t$
State covariance	Σ_t	P_t
Predicted mean	$\bar{\mu}_t$	$\hat{\mathbf{x}}_{t t-1}$
Predicted covariance	$\bar{\Sigma}_t$	$P_{t t-1}$
Linear state transition matrix	A_t	F_t
Linear control matrix	B_t	B_t
Linear observation matrix	C_t	H_t
Nonlinear motion model	$g(u_t, \mu_{t-1})$	$f(\mathbf{x}_{t-1}, \mathbf{u}_t)$
Nonlinear observation model	$h(\mu_t)$	$h(\mathbf{x}_t)$
Motion Jacobian	$G_t = \partial g / \partial x_{t-1}$	F_t
Observation Jacobian	$H_t = \partial h / \partial x_t$	H_t
Process noise covariance	R_t	Q_t
Measurement noise covariance	Q_t	R_t
Control input	u_t	$\mathbf{u}_t$
Measurement	z_t	$\mathbf{z}_t$
Belief	$\text{bel}(x_t)$	posterior $p(\mathbf{x}_t \mid \cdot)$
Kalman gain	K_t	K_t

Critical: R and Q are **swapped** between conventions. In this document R_t is process noise (motion uncertainty) and Q_t is measurement noise (sensor uncertainty). Memorise: *R for Robot motion, Q for Querying sensor*.

2 Introduction and Motivation

A mobile robot does not know where it is. It knows only what its sensors tell it, and every sensor lies in characteristic, predictable ways. Wheel encoders accumulate slip. Inertial sensors integrate bias into unbounded drift. Cameras lose features in textureless corridors. LiDARs degenerate in long featureless tunnels. GPS jitters by metres and disappears indoors.

Localisation is the problem of producing a single best estimate of the robot's pose given all of these flawed measurements. The Extended Kalman Filter is the workhorse algorithm for solving this in real time. When the system is approximately linear-Gaussian it is provably optimal in the minimum-mean-squared-error sense; even when mildly nonlinear, it is fast, well-understood, and good enough.

This guide assumes you have multi-sensor fusion already running on your platform. Goal: theoretical depth. You should finish understanding *why* every line of the EKF algorithm is the way it is, *what* the covariance matrix actually represents, *how* each sensor's physics produces the noise model the filter assumes, and *how* the dual-EKF architecture in `robot_localization` handles the local/global decomposition cleanly.

3 Mathematical Foundations

3.1 Random Variables and Probability Density Functions

A continuous random variable X on $\mathbb{R}^n$ is described by a probability density $p(x)$ satisfying:

$$p(x) \geq 0 \quad \forall x \in \mathbb{R}^n, \quad \int_{\mathbb{R}^n} p(x) dx = 1$$

Expectation:

$$\mathbb{E}[g(x)] = \int_{\mathbb{R}^n} g(x) p(x) dx$$

Mean and covariance:

$$\mu = \mathbb{E}[x] \in \mathbb{R}^n, \quad \Sigma = \mathbb{E}[(x - \mu)(x - \mu)^\top] \in \mathbb{R}^{n \times n}$$

3.2 The Multivariate Gaussian Distribution

The distribution that drives all of Kalman filtering:

$$p(x) = \det(2\pi\Sigma)^{-1/2} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right) \quad (1)$$

Two parameters (μ, Σ) describe it completely.

3.2.1 Properties of the Covariance Matrix

- **Symmetric:** $\Sigma = \Sigma^\top$
- **Positive semi-definite:** $v^\top \Sigma v \geq 0$ for all $v \in \mathbb{R}^n$
- **Diagonal entries:** $\Sigma_{ii} = \sigma_i^2 = \text{Var}(x_i)$
- **Off-diagonal entries:** $\Sigma_{ij} = \text{Cov}(x_i, x_j) = \rho_{ij} \sigma_i \sigma_j$, $\rho_{ij} \in [-1, 1]$.

For 2D position of a robot:

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

Level sets of $p(x)$ are ellipses defined by $(x - \mu)^\top \Sigma^{-1}(x - \mu) = c$. Principal axes align with eigenvectors of Σ , semi-axis lengths $\sqrt{c\lambda_i}$. Diagonal motion produces correlated x, y uncertainty — tilted ellipse.

3.2.2 The Mahalanobis Distance

$$d_M(x, \mu) = \sqrt{(x - \mu)^\top \Sigma^{-1}(x - \mu)}$$

Generalises Euclidean distance accounting for distribution shape. Two points equidistant in Mahalanobis sense have equal density. Invariant under linear coordinate transformation.

3.2.3 Why Gaussian?

1. **Closure under linear transforms.** If $x \sim \mathcal{N}(\mu, \Sigma)$ and $y = Ax + b$, then $y \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$.
2. **Closure under multiplication.** Product of Gaussian densities $\propto$ Gaussian density. Bayes update tractable.
3. **Central Limit Theorem.** Sums of many small independent effects $\rightarrow$ Gaussian. Sensor noise approximately Gaussian.

3.2.4 Product of Two Gaussians

$$\mathcal{N}(x; \mu_1, \Sigma_1) \cdot \mathcal{N}(x; \mu_2, \Sigma_2) \propto \mathcal{N}(x; \mu, \Sigma) \quad (2)$$

where

$$\Sigma = (\Sigma_1^{-1} + \Sigma_2^{-1})^{-1}, \quad \mu = \Sigma (\Sigma_1^{-1} \mu_1 + \Sigma_2^{-1} \mu_2)$$

Information form (Σ^{-1}) adds linearly. Posterior covariance always smaller (in PSD ordering) than either prior. This is exactly the Kalman update.

3.3 Conditional Gaussian

Joint Gaussian:

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}\right)$$

Conditional $p(x_1 | x_2)$ is Gaussian:

$$\mathbb{E}[x_1 | x_2] = \mu_1 + \Sigma_{12} \Sigma_{22}^{-1} (x_2 - \mu_2) \quad (3)$$

$$\text{Cov}(x_1 | x_2) = \Sigma_{11} - \Sigma_{12} \Sigma_{22}^{-1} \Sigma_{21} \quad (4)$$

This formula is the heart of the Kalman update. The Kalman gain $K_t = \Sigma_{12} \Sigma_{22}^{-1}$ is the regression coefficient of x_1 on x_2 .

4 The Bayes Filter

4.1 Belief and the Filtering Problem

The belief is the posterior over state given all observations and controls up to time t :

$$\text{bel}(x_t) = p(x_t | z_{1:t}, u_{1:t}) \quad (5)$$

Markov property assumed: given x_{t-1} , next state x_t is conditionally independent of all earlier history. Observations depend only on current state.

4.2 Recursive Filtering: Predict and Update

Bayes Filter Recursion

Prediction:

$$\overline{\text{bel}}(x_t) = \int p(x_t | x_{t-1}, u_t) \text{bel}(x_{t-1}) dx_{t-1}$$

Correction:

$$\text{bel}(x_t) = \eta p(z_t | x_t) \overline{\text{bel}}(x_t)$$

η normalising constant.

4.2.1 Derivation

Prediction. Total probability:

$$p(x_t | z_{1:t-1}, u_{1:t}) = \int p(x_t | x_{t-1}, z_{1:t-1}, u_{1:t}) p(x_{t-1} | z_{1:t-1}, u_{1:t-1}) dx_{t-1}$$

By Markov, $p(x_t | x_{t-1}, \cdot) = p(x_t | x_{t-1}, u_t)$. Second factor is $\text{bel}(x_{t-1})$.

Correction. Bayes rule:

$$p(x_t \mid z_t, z_{1:t-1}, u_{1:t}) = \frac{p(z_t \mid x_t) p(x_t \mid z_{1:t-1}, u_{1:t})}{p(z_t \mid z_{1:t-1}, u_{1:t})}$$

Likelihood depends only on x_t by Markov. Denominator constant in x_t .

4.2.2 From Bayes to Kalman

Bayes filter universal but integral intractable in general. For *linear-Gaussian* systems, integral has closed form: belief stays Gaussian forever, parametrised by (μ_t, Σ_t) only. That closed form is the Kalman filter.

5 The Linear Kalman Filter

5.1 System Model

Linear-Gaussian discrete-time system:

$$x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_t) \quad (6)$$

$$z_t = C_t x_t + \delta_t, \quad \delta_t \sim \mathcal{N}(0, Q_t) \quad (7)$$

- $x_t \in \mathbb{R}^n$: state vector
- $u_t \in \mathbb{R}^m$: control input
- $z_t \in \mathbb{R}^k$: measurement
- A_t : state transition matrix
- B_t : control matrix
- C_t : observation matrix mapping state to measurement space
- ε_t, δ_t : process and measurement noise — mutually independent, white, zero-mean Gaussian
- R_t : process noise covariance
- Q_t : measurement noise covariance

5.2 KF Algorithm

Algorithm: Kalman_filter

Inputs: $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$

Prediction step:

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \quad (\text{predicted mean})$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R_t \quad (\text{predicted covariance})$$

Correction step:

$$K_t = \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1} \quad (\text{Kalman gain})$$

$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \quad (\text{updated mean})$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \quad (\text{updated covariance})$$

Return μ_t, Σ_t

5.3 Derivation of Prediction Step

Given $x_{t-1} \sim \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$ and linear dynamics with Gaussian noise:

$$x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t$$

Mean by linearity:

$$\mathbb{E}[x_t] = A_t \mathbb{E}[x_{t-1}] + B_t u_t = A_t \mu_{t-1} + B_t u_t \equiv \bar{\mu}_t$$

Covariance by affine-Gaussian rule and noise independence:

$$\text{Cov}(x_t) = A_t \Sigma_{t-1} A_t^\top + R_t \equiv \bar{\Sigma}_t$$

5.4 Derivation of Correction Step

Joint distribution of state and measurement, given prediction:

$$\begin{bmatrix} x_t \\ z_t \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \bar{\mu}_t \\ C_t \bar{\mu}_t \end{bmatrix}, \begin{bmatrix} \bar{\Sigma}_t & \bar{\Sigma}_t C_t^\top \\ C_t \bar{\Sigma}_t & C_t \bar{\Sigma}_t C_t^\top + Q_t \end{bmatrix}\right)$$

Apply conditional Gaussian formula with $x_1 = x_t$, $x_2 = z_t$:

$$\mu_t = \bar{\mu}_t + \underbrace{\bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}}_{K_t} (z_t - C_t \bar{\mu}_t) \quad (8)$$

$$\Sigma_t = \bar{\Sigma}_t - K_t C_t \bar{\Sigma}_t = (I - K_t C_t) \bar{\Sigma}_t \quad (9)$$

The Kalman gain is the regression coefficient of x_t on z_t from the joint Gaussian.

5.5 The Kalman Gain: Intuition

Scalar (1D) form with $C = 1$:

$$K_t = \frac{\bar{\Sigma}_t}{\bar{\Sigma}_t + Q_t}, \quad \mu_t = \bar{\mu}_t + K_t (z_t - \bar{\mu}_t)$$

Rewriting:

$$\mu_t = (1 - K_t) \bar{\mu}_t + K_t z_t$$

Weighted average of prediction and measurement, weighted by inverse-variance.

Limit	Kalman gain	Behaviour
$Q_t \rightarrow 0$ (perfect sensor)	$K_t \rightarrow 1$	Trust measurement entirely
$\bar{\Sigma}_t \rightarrow 0$ (perfect prediction)	$K_t \rightarrow 0$	Trust prediction entirely
$Q_t = \bar{\Sigma}_t$ (equal)	$K_t = 0.5$	Average them

Posterior variance:

$$\Sigma_t = (1 - K_t) \bar{\Sigma}_t = \frac{\bar{\Sigma}_t Q_t}{\bar{\Sigma}_t + Q_t}$$

Always less than either input variance — measurement always reduces uncertainty.

5.6 Joseph Form: Numerical Stability

The covariance update $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$ is mathematically correct but numerically dangerous: floating-point round-off can cause Σ_t to lose symmetry or positive-definiteness. **Joseph form** is algebraically equivalent but robust:

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t (I - K_t C_t)^\top + K_t Q_t K_t^\top \quad (10)$$

Symmetric by construction. Remains positive definite under any choice of K_t . Always use in production.

5.7 Optimality

KF is the linear MMSE estimator. Among all linear estimators of x_t from $\{z_{1:t}, u_{1:t}\}$, minimises:

$$\mathbb{E}[\|x_t - \mu_t\|^2]$$

For Gaussian noise, also unconstrained MMSE estimator (no nonlinear estimator does better). For non-Gaussian noise with same covariance, remains the BLUE (Best Linear Unbiased Estimator).

6 The Extended Kalman Filter

6.1 Why the Linear KF Fails for Robots

Real robot motion is nonlinear. Differential-drive kinematics:

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix} = \begin{bmatrix} x_t + v \cos(\theta_t) \Delta t \\ y_t + v \sin(\theta_t) \Delta t \\ \theta_t + \omega \Delta t \end{bmatrix}$$

Trigonometric functions of θ_t couple state components nonlinearly. A Gaussian belief pushed through a nonlinear function does *not* produce a Gaussian output — output is banana-shaped, skewed, or worse.

Key Insight

The KF assumes Gaussian-in $\rightarrow$ Gaussian-out, true only for linear systems. EKF approximates nonlinear systems as locally linear, then applies KF machinery to linearised system.

6.2 First-Order Taylor Expansion

For a smooth nonlinear function $g : \mathbb{R}^n \rightarrow \mathbb{R}^n$ around operating point μ :

$$g(x) = g(\mu) + \underbrace{\frac{\partial g}{\partial x} \Big|_{\mu}}_{G_t = \text{Jacobian}} (x - \mu) + \underbrace{\frac{1}{2} (x - \mu)^\top \mathbf{H}_g (x - \mu)}_{\text{quadratic remainder}} + O(\|x - \mu\|^3)$$

EKF discards quadratic and higher terms. Approximation exact at μ , degrades farther away. **Source of EKF approximation error:** when true state far from linearisation point, linear approximation poor, filter can diverge.

6.3 The Jacobian Matrix

Jacobian of $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is $m \times n$ matrix of partial derivatives:

$$G = \frac{\partial g}{\partial x} = \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \dots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \dots & \frac{\partial g_m}{\partial x_n} \end{bmatrix}$$

6.4 Differential-Drive Motion Jacobian

State $x = [x, y, \theta]^\top$, control $u = [v, \omega]^\top$, motion model:

$$g(u, \mu) = \begin{bmatrix} \mu_x + v \cos(\mu_\theta) \Delta t \\ \mu_y + v \sin(\mu_\theta) \Delta t \\ \mu_\theta + \omega \Delta t \end{bmatrix}$$

State Jacobian (linearise w.r.t. x):

$$G_t = \left. \frac{\partial g}{\partial x} \right|_{\mu_{t-1}} = \begin{bmatrix} 1 & 0 & -v \sin(\mu_\theta) \Delta t \\ 0 & 1 & v \cos(\mu_\theta) \Delta t \\ 0 & 0 & 1 \end{bmatrix} \quad (11)$$

Control Jacobian (linearise w.r.t. u , denoted V_t in some texts):

$$V_t = \left. \frac{\partial g}{\partial u} \right|_{\mu_{t-1}} = \begin{bmatrix} \cos(\mu_\theta) \Delta t & 0 \\ \sin(\mu_\theta) \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \quad (12)$$

The only nonlinearity: heading-position coupling visible in column 3 of G_t . Small change in θ rotates subsequent translation. G_t depends on current estimate μ_θ and control v — recompute every time step.

6.5 Observation Jacobian Examples

GPS measuring position: state $x = [x, y, \theta, \dot{x}, \dot{\theta}]^\top$, measurement $z = [x, y]^\top$:

$$H_t = \frac{\partial h}{\partial x} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Linear observation, H_t constant.

Range-bearing to landmark (ℓ_x, ℓ_y):

$$h(\mu) = \begin{bmatrix} \sqrt{(\ell_x - \mu_x)^2 + (\ell_y - \mu_y)^2} \\ \text{atan2}(\ell_y - \mu_y, \ell_x - \mu_x) - \mu_\theta \end{bmatrix}$$

Let $q = (\ell_x - \mu_x)^2 + (\ell_y - \mu_y)^2$, $r = \sqrt{q}$:

$$H_t = \frac{1}{q} \begin{bmatrix} -r(\ell_x - \mu_x) & -r(\ell_y - \mu_y) & 0 & 0 & 0 \\ (\ell_y - \mu_y) & -(\ell_x - \mu_x) & -q & 0 & 0 \end{bmatrix}$$

Genuine nonlinear observation — H_t depends on current pose estimate.

6.6 Full EKF Algorithm

Algorithm: Extended_Kalman_filter

Inputs: $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$

Compute Jacobians:

$$G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}, \quad H_t = \frac{\partial h(\bar{\mu}_t)}{\partial x_t}$$

Prediction step:

$$\bar{\mu}_t = g(u_t, \mu_{t-1}) \quad (\text{nonlinear propagation})$$

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t \quad (\text{linearised covariance})$$

Correction step:

$$K_t = \bar{\Sigma}_t H_t^\top (H_t \bar{\Sigma}_t H_t^\top + Q_t)^{-1} \quad (\text{Kalman gain})$$

$$\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t)) \quad (\text{updated mean})$$

$$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t \quad (\text{updated covariance})$$

Return μ_t, Σ_t

Asymmetry: Mean propagated through true nonlinear g , covariance through linearised G_t . This asymmetry is the EKF approximation. Unscented KF (UKF) handles differently — propagates sigma points through g , reconstructs mean/covariance, no Jacobians.

6.7 Worked Example: Differential-Drive EKF One Step

State $\mu = [x, y, \theta]^\top$, control $u = [v, \omega]^\top$.

Setup: $\mu_{t-1} = [0, 0, 0]^\top$, $\Sigma_{t-1} = 0.1 I_3$, $u_t = [1.0, 0.1]^\top$, $\Delta t = 0.1$ s. GPS reading $z_t = [0.12, 0.001]^\top$ with $Q_t = 0.05 I_2$. Process noise $R_t = \text{diag}(0.001, 0.001, 0.0005)$.

Compute Jacobians: at μ_{t-1} :

$$G_t = \begin{bmatrix} 1 & 0 & -1.0 \cdot \sin(0) \cdot 0.1 \\ 0 & 1 & 1.0 \cdot \cos(0) \cdot 0.1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0.1 \\ 0 & 0 & 1 \end{bmatrix}$$

GPS observation Jacobian:

$$H_t = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Predict mean:

$$\bar{\mu}_t = g(u_t, \mu_{t-1}) = \begin{bmatrix} 0 + 1.0 \cdot \cos(0) \cdot 0.1 \\ 0 + 1.0 \cdot \sin(0) \cdot 0.1 \\ 0 + 0.1 \cdot 0.1 \end{bmatrix} = \begin{bmatrix} 0.10 \\ 0 \\ 0.01 \end{bmatrix}$$

Predict covariance:

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t \approx \begin{bmatrix} 0.101 & 0 & 0 \\ 0 & 0.111 & 0.01 \\ 0 & 0.01 & 0.1005 \end{bmatrix}$$

Innovation:

$$z_t - h(\bar{\mu}_t) = \begin{bmatrix} 0.12 \\ 0.001 \end{bmatrix} - \begin{bmatrix} 0.10 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.02 \\ 0.001 \end{bmatrix}$$

Innovation covariance:

$$S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t = \begin{bmatrix} 0.151 & 0 \\ 0 & 0.161 \end{bmatrix}$$

Kalman gain:

$$K_t = \bar{\Sigma}_t H_t^\top S_t^{-1} \approx \begin{bmatrix} 0.669 & 0 \\ 0 & 0.689 \\ 0 & 0.062 \end{bmatrix}$$

Update mean:

$$\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t)) \approx \begin{bmatrix} 0.113 \\ 0.0007 \\ 0.0101 \end{bmatrix}$$

Update covariance: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$. Position variances shrink to ≈ 0.033 ; heading variance $\Sigma_{\theta\theta}$ stays at ≈ 0.1 (GPS doesn't measure θ directly, but cross-correlation through $\Sigma_{y\theta}$ pulls it slightly).

GPS measurement pulled x from 0.10 to 0.113. Heading uncertainty largely untouched because GPS does not observe θ . Filter learned position from GPS but not orientation.

6.8 Practical Issues

6.8.1 Angle Wrapping

Angles live on circle, not line. Innovation:

$$y_\theta = z_\theta - \bar{\mu}_\theta$$

can produce $\pm 2\pi$ jumps when angles cross wrap-around. Fix:

$$y_\theta \leftarrow \text{atan2}(\sin(y_\theta), \cos(y_\theta))$$

forcing $y_\theta \in (-\pi, \pi]$. Forgetting this is the most common cause of EKF divergence in practice.

6.8.2 Quaternions for 3D Orientation

For 3D systems, Euler angles suffer gimbal lock at $\theta = \pm 90^\circ$. Use unit quaternions $q = [q_w, q_x, q_y, q_z]^\top$, $\|q\| = 1$. Kinematics:

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix}$$

where $\otimes$ is quaternion product. Discretise:

$$q_{t+1} = q_t \otimes q_\Delta, \quad q_\Delta = \begin{bmatrix} \cos(\|\omega\| \Delta t / 2) \\ \sin(\|\omega\| \Delta t / 2) \hat{\omega} \end{bmatrix}$$

followed by renormalisation. Error-state EKF (ESEKF) handles over-parametrisation cleanly by tracking 3-parameter local error around nominal quaternion.

6.8.3 Uncertainty “Breath”

During pure prediction (dead reckoning, no measurements):

$$\bar{\Sigma}_{t+k} = G^k \Sigma_t (G^k)^\top + \sum_{i=0}^{k-1} G^i R (G^i)^\top$$

Σ grows — uncertainty expands. Measurement arrives, Σ shrinks. Filter “breathes”. With no measurements, position uncertainty grows unboundedly — dead reckoning drift.

6.8.4 Mahalanobis Gating for Outlier Rejection

Measurement plausible if Mahalanobis distance to prediction small:

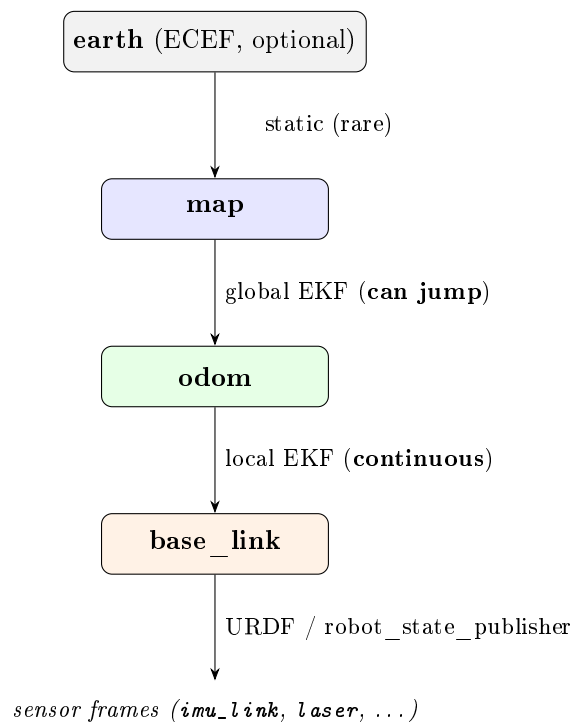
$$d_M^2 = (z_t - h(\bar{\mu}_t))^\top S_t^{-1} (z_t - h(\bar{\mu}_t)) \sim \chi_k^2$$

where k is measurement dimension, $S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t$. Reject measurements with $d_M^2 > \chi_{k,0.99}^2$ — outliers (GPS multipath, false LiDAR matches). Essential for robust operation.

7 Local and Global Frames: The Dual EKF Architecture

7.1 ROS Frame Hierarchy (REP-105)

ROS REP-105 defines a strict transformation hierarchy:



Frame definitions:

- **earth**: ECEF (Earth-Centered, Earth-Fixed). Used for multi-robot or surveying. Usually omitted.
- **map**: world-fixed reference. Globally consistent. Origin chosen at startup or pre-defined. ENU axes (X-East, Y-North, Z-Up) by convention.
- **odom**: world-fixed reference for the robot. Drifts over time but **continuous**. Origin at robot’s startup pose typically.

- **base_link**: rigidly attached to robot body. Convention: X forward, Y left, Z up.
- Sensor frames: rigidly attached to body via static transforms from URDF.

7.2 Why Two Frames? Why Two EKFs?

ROS Navigation Stack imposes a hard constraint:

Navigation Constraint

The transform `odom → base_link` must be **continuous** and **monotonic**. No jumps. Ever.

Local planner integrates pose forward to compute trajectories and velocity commands. Discontinuity in **base_link** pose would make controller believe robot teleported — catastrophic control commands emitted.

But absolute references like GPS *do* produce position corrections, sometimes large. Need somewhere to absorb these jumps. Solution: **decouple the transform chain**.

Transform	Continuous?	Purpose
<code>odom → base_link</code>	Yes (smooth)	Local planning, control
<code>map → odom</code>	No (can jump)	Global path planning

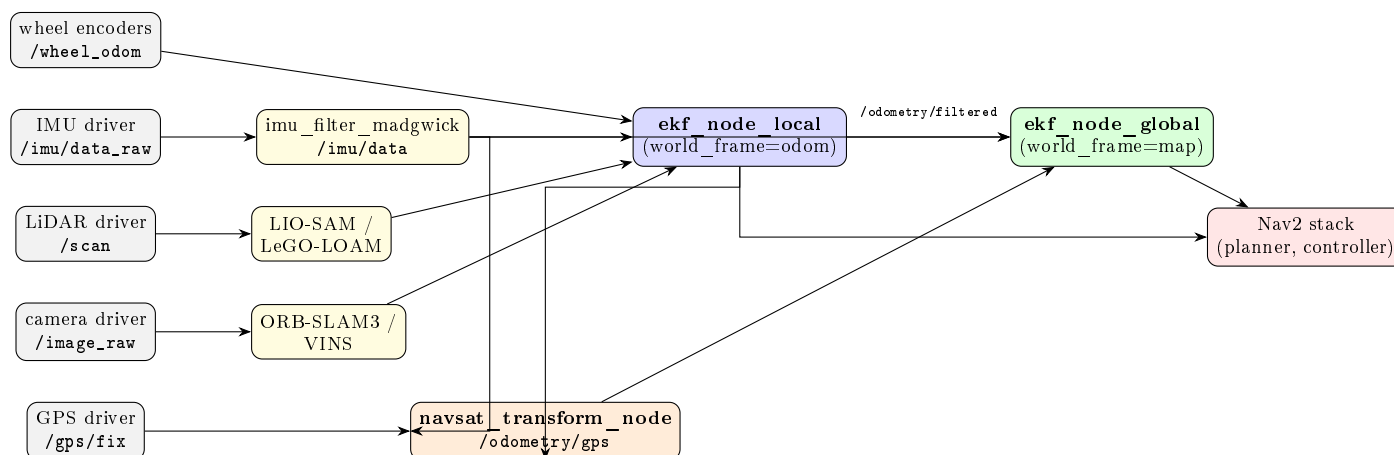
When GPS provides correction, only `map → odom` jumps; `odom → base_link` unaffected. Robot pose in map frame = composition of both transforms, absorbs correction cleanly:

$$T_{\text{map}}^{\text{base}} = T_{\text{map}}^{\text{odom}} \cdot T_{\text{odom}}^{\text{base}}$$

When GPS jumps, $T_{\text{map}}^{\text{odom}}$ adjusts to compensate for accumulated $T_{\text{odom}}^{\text{base}}$ drift, leaving $T_{\text{map}}^{\text{base}}$ globally accurate.

8 robot_localization Architecture

8.1 Node Topology



8.2 Node-by-Node Walkthrough

8.2.1 ekf_node_local (Local EKF)

Package: robot_localization

Executable: ekf_node

Role: Fuse continuous-only sensors into smooth local pose estimate.

Subscribes to:

- /wheel_odom (nav_msgs/Odometry) — wheel encoder velocities
- /imu/data (sensor_msgs/Imu) — filtered IMU
- /lidar_odom (nav_msgs/Odometry) — LiDAR odometry (optional)
- /visual_odom (nav_msgs/Odometry) — visual odometry (optional)

Publishes:

- /odometry/filtered (nav_msgs/Odometry) — fused state, in odom frame
- TF: odom → base_link (continuous)
- /diagnostics (diagnostic_msgs/DiagnosticArray)

Configuration parameters (key):

```

1 ekf_node_local:
2   ros__parameters:
3     frequency: 50.0                # filter publish rate Hz
4     sensor_timeout: 0.1            # s before marking sensor offline
5     two_d_mode: true               # planar robot (zero z, roll, pitch)
6     publish_tf: true
7     publish_acceleration: false
8
9     map_frame: map
10    odom_frame: odom
11    base_link_frame: base_link
12    world_frame: odom               # KEY: local EKF in odom frame
13
14    # Sensor 0: wheel odometry (use velocities only)
15    odom0: /wheel_odom
16    odom0_config: [false, false, false, # x, y, z position
17                  false, false, false, # roll, pitch, yaw
18                  true, false, false, # vx, vy, vz
19                  false, false, true, # vroll, vpitch, vyaw
20                  false, false, false] # ax, ay, az
21    odom0_differential: false
22    odom0_relative: false
23    odom0_queue_size: 10
24
25    # Sensor 1: IMU (orientation + angular velocities)
26    imu0: /imu/data
27    imu0_config: [false, false, false,
28                 true, true, false, # roll, pitch (no yaw if no mag)
29                 false, false, false,
30                 true, true, true, # all angular velocities
31                 true, true, false] # planar accels
32    imu0_differential: false
33    imu0_remove_gravitational_acceleration: true
34
35    # Process noise (R_t in Thrun notation)
36    process_noise_covariance: [0.05, 0, 0, ..., # x
37                               0, 0.05, 0, ..., # y

```



```

38         ...]
39     initial_estimate_covariance: [1e-9, 0, 0, ...]

```

Internal state vector (15D):

$$\mu_t = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}, \ddot{x}, \ddot{y}, \ddot{z}]^\top$$

In `two_d_mode`: `true`, components $z, \phi, \theta, \dot{z}, \dot{\phi}, \dot{\theta}, \ddot{z}$ are clamped to zero.

8.2.2 ekf_node_global (Global EKF)

Same executable, different config. Sets `world_frame`: `map`. The role differs sharply between the canonical `robot_localization` tutorial pattern and the architecture used in this project.

Canonical pattern (Tom Moore reference example). Both EKFs subscribe to the *same raw sensors* (wheel odom, IMU). Global EKF additionally subscribes to `/odometry/gps` from `navsat_transform_node`. GPS provides absolute position correction; map frame stays globally anchored.

Project pattern (Husarion Lynx, this codebase). Global EKF **deliberately excludes GPS**. Both EKFs receive identical sensor inputs (wheel + VO + LiDAR + IMU), differing only in `world_frame`. GPS still flows through `navsat_transform_node` and is published on `/odometry/gps` for analysis, but it is *not* listed in EKF #2's sensor inputs. The reason for this departure is detailed in the next subsection (covariance cross-term leak); briefly, single-antenna GPS noise injected into a filter without a strong yaw observation produces a $\sim 3.56^\circ/\text{s}$ yaw drift in the map frame.

Subscribes to (this project):

- `/diff_drive_controller/odom` — raw wheel odometry (twist only)
- `/imu/data` — IMU (orientation, angular velocity, acceleration)
- `/odometry/visual` — RTAB-Map visual odometry
- `/odometry/lidar` — ICP LiDAR odometry
- (NOT `/odometry/gps`)

Publishes:

- `/odometry/filtered_map` — fused state in map frame
- TF: `map` $\rightarrow$ `odom` (**near-identity** in this project; can jump in canonical pattern)

Config delta (canonical pattern):

```

1 ekf_node_global:
2   ros__parameters:
3     world_frame: map
4
5     odom0: /wheel/odometry                                # SAME raw sensor as local EKF
6     odom0_config: [false, false, false,
7                    false, false, false,
8                    true,  false, false,
9                    false, false, true,
10                   false, false, false]
11

```

```

12 imu0: /imu/data # SAME raw sensor as local EKF
13
14 odom1: /odometry/gps # only canonical: enable GPS here
15 odom1_config: [true, true, false,
16               false, false, false,
17               false, false, false,
18               false, false, false,
19               false, false, false]

```

Config delta (project, GPS excluded):

```

1 ekf_node_global:
2   ros__parameters:
3     world_frame: map
4
5     odom0: /diff_drive_controller/odom # raw wheel
6     odom1: /odometry/visual # raw VO
7     odom2: /odometry/lidar # raw LO
8     imu0: /imu/data # raw IMU
9     # GPS is INTENTIONALLY OMITTED - see Section on covariance leak

```

Common Misconception

A frequent misconfiguration is feeding `/odometry/filtered` (local EKF output) into the global EKF as a sensor input. This creates a chain where the global filter consumes the local filter's already-fused estimate. Per Tom Moore's reference example (`cra-ros-pkg/robot_localization/params/dual_ekf_navsat_example.yaml`), both EKFs must subscribe to the *same raw sensors independently*, in parallel. Chaining produces:

- Double-counting of measurement information
- Covariance accounting that diverges from optimal
- Feedback loop with `navsat_transform_node` (which already reads `/odometry/filtered`)

8.2.3 navsat_transform_node

Package: `robot_localization`

Executable: `navsat_transform_node`

Role: Convert NavSatFix (lat, lon, alt) into Odometry in map frame.

Subscribes to:

- `/gps/fix` (`sensor_msgs/NavSatFix`) — raw GPS
- `/imu/data` (`sensor_msgs/Imu`) — for initial heading
- `/odometry/filtered` — fused robot pose for transform anchoring

Publishes:

- `/odometry/gps` (`nav_msgs/Odometry`) — GPS measurement in map frame, ready for global EKF
- `/gps/filtered` (`sensor_msgs/NavSatFix`) — GPS in lat/lon corresponding to filtered state

Internal pipeline:

1. **LL2UTM conversion:** latitude/longitude $\rightarrow$ UTM (Universal Transverse Mercator) coordinates. UTM is a metric, locally Cartesian projection. The Earth surface is divided into 60 zones of 6° longitude. Inside each zone, conversion is essentially:

$$N = k_0 A_0 + (\text{series in } \phi, \lambda)$$

$$E = k_0 [\text{series in } \phi, \lambda] + 500,000$$

where $k_0 = 0.9996$ is the central meridian scale factor. Output: $(E_{\text{UTM}}, N_{\text{UTM}})$ in metres.

2. **Datum establishment:** on first GPS fix (or from `datum` parameter), record the initial UTM coordinates as origin:

$$(E_0, N_0) = (E_{\text{UTM}}^{(0)}, N_{\text{UTM}}^{(0)})$$

3. **Local Cartesian offset:**

$$(x_{\text{local}}, y_{\text{local}}) = (E_{\text{UTM}} - E_0, N_{\text{UTM}} - N_0)$$

4. **Heading rotation:** map frame's $+x$ axis may not coincide with UTM East. Rotate by initial yaw ψ_0 :

$$\begin{bmatrix} x_{\text{map}} \\ y_{\text{map}} \end{bmatrix} = \begin{bmatrix} \cos \psi_0 & \sin \psi_0 \\ -\sin \psi_0 & \cos \psi_0 \end{bmatrix} \begin{bmatrix} x_{\text{local}} \\ y_{\text{local}} \end{bmatrix}$$

ψ_0 obtained from magnetometer heading at startup, or set explicitly via `yaw_offset` param.

5. **Publish:** $(x_{\text{map}}, y_{\text{map}})$ as `Odometry` message with covariance computed from GPS fix's reported HDOP/VDOP.

Configuration:

```

1 navsat_transform_node:
2   ros__parameters:
3     frequency: 30.0
4     delay: 3.0                                # wait for sensor fixes to stabilise
5     magnetic_declination_radians: 0.0          # local magnetic declination (look up
6     yaw_offset: 0.0                            # IMU 0 yaw vs map +X axis offset
7     zero_altitude: true                       # ignore altitude (planar robot)
8     publish_filtered_gps: true
9     use_odometry_yaw: false                   # use IMU yaw, not odom yaw, for
10    datum                                       datum
11    wait_for_datum: false                     # auto-set datum on first fix
12    # Or explicit datum:
13    # datum: [55.944904, -3.186693, 0.0] # lat, lon, yaw

```

Magnetic declination: the angle between true north and magnetic north at your location. Varies geographically (-20° to $+20^\circ$ globally). Look up via NOAA's geomagnetic calculator.

8.2.4 imu_filter_madgwick (Pre-Filter)

Package: `imu_filter_madgwick`

Role: Convert raw accelerometer + gyroscope (+ magnetometer) into a stable orientation estimate. Subscribes to `/imu/data_raw`, publishes `/imu/data` with orientation field populated.

Madgwick filter is a complementary-filter family algorithm: gyro integration with gradient-descent correction toward gravity (for roll/pitch) and magnetic field (for yaw). Cheaper than EKF, robust, used widely.

This pre-filter exists because most cheap IMUs publish only raw rates and accelerations — no orientation. The downstream EKF expects orientation in the IMU message.

8.2.5 LiDAR Odometry Node (e.g., LIO-SAM)

External node (not part of `robot_localization`). Performs scan-matching + IMU preintegration + factor-graph optimisation. Publishes `nav_msgs/Odometry` with 6-DOF pose and twist + covariance.

8.3 Topic and TF Flow Summary

Topic / TF	Source	Sink
<code>/diff_drive_controller/odom</code>	wheel driver	both EKFs (parallel)
<code>/imu/data_raw</code>	IMU driver	madgwick filter
<code>/imu/data</code>	madgwick	both EKFs (parallel), navsat
<code>/gps/fix</code>	GPS driver	navsat (canonical also $\rightarrow$ global EKF)
<code>/scan</code> or <code>/points</code>	LiDAR driver	LiDAR odom node
<code>/odometry/lidar</code>	LiDAR odom	both EKFs (parallel)
<code>/odometry/visual</code>	VO node	both EKFs (parallel)
<code>/odometry/filtered</code>	local EKF	navsat (datum bootstrap), Nav2
<code>/odometry/gps</code>	navsat	global EKF (canonical) / analysis only (project)
<code>/odometry/filtered_map</code>	global EKF	Nav2 (planner)
TF <code>odom $\rightarrow$ base_link</code>	local EKF	TF tree (continuous)
TF <code>map $\rightarrow$ odom</code>	global EKF	TF tree (jumps in canonical, $\approx I$ in project)
TF <code>base_link $\rightarrow$ sensor</code>	URDF / static	TF tree

8.4 Why Each Sensor Goes Where

Canonical pattern:

- **Wheel odom $\rightarrow$ both EKFs (parallel):** high-rate smooth velocities. Same raw msg fed to both.
- **IMU $\rightarrow$ both EKFs (parallel):** high-rate orientation + angular velocity. Same raw msg fed to both.
- **LiDAR/Visual odom $\rightarrow$ both EKFs:** continuous pose deltas, both filters benefit.
- **GPS $\rightarrow$ global EKF only:** GPS jumps would break local continuity. Goes only to global via navsat.

Project pattern (this codebase):

- **Wheel, IMU, VO, LiDAR $\rightarrow$ both EKFs (parallel):** identical sensor consensus across both filters.
- **GPS $\rightarrow$ navsat _transform only, NOT into either EKF:** GPS published on `/odometry/gps` but excluded from both EKF input lists. Available for analysis, ignored for fusion.

The project pattern produces `map  $\rightarrow$  odom` $\approx$ identity transform (typically ± 0.1 m translation, ± 1 rotation over a 2-minute run) because both filters process identical inputs and converge to nearly identical state estimates.

8.5 The Covariance Cross-Term Leak (Why GPS is Excluded in this Project)

This is the core engineering reason behind the project's deliberate departure from canonical dual-EKF + NavSat. The phenomenon is subtle but has measurable consequences (3.56°/s map frame yaw drift in this project's earlier configuration with GPS enabled).

8.5.1 The Mechanism

The EKF state covariance Σ is a dense 15×15 matrix — not diagonal. Off-diagonal entries $\Sigma_{x,\psi}$ and $\Sigma_{y,\psi}$ encode statistical correlation: “if I learn something about x , what does it tell me about yaw ψ ?”

These cross-terms become non-zero whenever the robot moves with $\omega \neq 0$ because the motion-model Jacobian G_t has a non-zero $\partial g_x / \partial \psi$ entry:

$$G_t = \begin{bmatrix} 1 & 0 & -v \sin(\mu_\theta) \Delta t \\ 0 & 1 & v \cos(\mu_\theta) \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

Predicted covariance $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$ inherits cross-terms from this Jacobian. After any curved motion, $\Sigma_{x,\psi}, \Sigma_{y,\psi} \neq 0$.

8.5.2 The Leak Path

When a GPS measurement arrives, H_{GPS} selects only $[x, y]$. But the Kalman gain $K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$ has a row for *every* state component, including yaw. The yaw update entry:

$$K_{\psi,x} = \frac{\Sigma_{\psi,x}}{\Sigma_{xx} + Q_{\text{GPS}}}$$

GPS noise $\sigma_{\text{GPS}} \approx 0.5$ m produces innovation $\nu_x \neq 0$ on every cycle. Yaw correction:

$$\Delta\psi = K_{\psi,x} \nu_x + K_{\psi,y} \nu_y$$

GPS never observed yaw, but yaw moved. Each fix injects a small angular correction.

8.5.3 Compounding

The Joseph form covariance update reduces but does not eliminate the cross-term:

$$\Sigma_{\psi,x}^{\text{new}} = \Sigma_{\psi,x} - K_{\psi,x} \Sigma_{xx}$$

Then the next predict step regrows it via G_t . Subsequent VO or LiDAR updates — which observe both position and yaw — amplify the leak by simultaneously correcting position (matching the GPS-shifted estimate) and re-using the now-shaped covariance.

Result: yaw drift compounds at $\approx 3.56/\text{s}$ in the configured system.

8.5.4 Numerical Walk-Through (Project Values)

State after some driving (illustrative):

$$\Sigma_{xx} = 0.30 \text{ m}^2, \quad \Sigma_{\psi\psi} = 0.02 \text{ rad}^2, \quad \Sigma_{x,\psi} = 0.04$$

GPS innovation $\nu_x = 0.5$ m (typical noise spike), $Q_{\text{GPS}} = 0.26 \text{ m}^2$:

$$K_{\psi,x} = \frac{0.04}{0.30 + 0.26} \approx 0.071$$

$$\Delta\psi = 0.071 \cdot 0.5 \approx 0.036 \text{ rad} \approx 2^\circ$$

Single GPS fix moved yaw by 2° via cross-term coupling. At 10 Hz GPS, even with partial randomness in direction, sustained walk emerges.

8.5.5 Why the Canonical Tutorial Doesn't Suffer This

Tom Moore's canonical configuration assumes a strong absolute yaw observation, typically from:

- Dual-antenna RTK GPS providing measured heading
- Reliable magnetometer with proper hard/soft-iron calibration
- Visual landmarks with known orientation (rare)

Under those conditions, an absolute yaw measurement is provided each cycle. Its measurement-noise covariance Q_ψ is small. The Kalman gain heavily weights this direct yaw observation, dominating the indirect leak from GPS cross-terms. Yaw stays bounded.

In this project, the simulated IMU has *no magnetometer*; the system has *no dual-antenna RTK*. Yaw is integrated from gyro only — no absolute observation. Nothing fights the leak.

8.5.6 The Sensor Consensus Solution

By feeding both EKFs identical sensor sets — and *omitting GPS from EKF #2* — the project enforces:

$$\mu_t^{(\text{global})} \approx \mu_t^{(\text{local})} \implies T_{\text{map}}^{\text{odom}} \approx I \quad (13)$$

The map $\rightarrow$ odom transform stays near-identity. No GPS-driven cross-term leakage. Map frame angularly stable.

Trade-off: The map frame is no longer GPS-anchored to absolute world coordinates. The robot's map-frame pose drifts with wheel/IMU drift over arbitrary distances. For indoor/short-range outdoor missions where global georeferencing is not required, this is acceptable.

8.5.7 Alternatives if Global Anchoring is Required

If you need both global anchoring *and* no leak:

1. **Add a strong yaw observation:** dual-antenna RTK, calibrated magnetometer, or sun sensor.
2. **Diagonalise Σ after GPS update:** zero out $\Sigma_{x,\psi}, \Sigma_{y,\psi}$ post-correction. Hacky but eliminates leak. Requires custom EKF code (not available in stock `robot_localization`).
3. **Loosely-coupled GPS in a separate filter:** run a third filter that consumes only EKF #2 output and GPS, then publish a `utm` $\rightarrow$ `map` transform without polluting EKF #2's state.
4. **Periodic re-anchoring:** when GPS confidence is high (e.g., RTK fix), do a one-shot pose reset of EKF #2 instead of continuous fusion.

Empirical Evidence in this Project

Earlier configuration (GPS in EKF #2): map frame yaw drifted at $3.56^\circ/\text{s}$ sustained, accumulating to over 200° in 2 minutes — catastrophic for navigation.

Current configuration (GPS removed from EKF #2): map $\rightarrow$ odom transform stays within ± 0.1 m and ± 1 over a 2-minute run.

The diagnosis trace:

1. Disable GPS in EKF #2 $\rightarrow$ drift disappears.
2. Re-enable GPS $\rightarrow$ drift returns.
3. Disable VO and LiDAR (only wheel + IMU + GPS) $\rightarrow$ drift much reduced because no second update path amplifies the cross-term.

The cross-term leak is the cause; sensor consensus is the cure.

9 The EKF Execution Loop

9.1 Local EKF Pseudocode

```

1  # Initialization
2  mu = np.zeros(15)                # state mean
3  Sigma = initial_covariance()    # state covariance
4  last_t = ros_time_now()
5
6  # Main loop @ frequency parameter (e.g. 50 Hz)
7  while node_ok():
8      now = ros_time_now()
9      dt = now - last_t
10     last_t = now
11
12     # === PREDICT ===
13     # Apply nonlinear motion model g
14     mu_bar = g(mu, dt)
15     G = jacobian_g(mu, dt)
16     Sigma_bar = G @ Sigma @ G.T + R * dt  # process noise R scaled by dt
17
18     # === CORRECT (per pending measurement) ===
19     for sensor_msg in get_pending_measurements_in_time_order():
20         z = sensor_msg.measurement
21         Q = sensor_msg.covariance
22         h = sensor_msg.h_function
23         H = sensor_msg.H_jacobian(mu_bar)
24
25         # Innovation
26         innov = z - h(mu_bar)
27         innov = wrap_angles(innov)        # critical for orientation
28
29         # Mahalanobis gating
30         S = H @ Sigma_bar @ H.T + Q
31         d2 = innov.T @ np.linalg.inv(S) @ innov
32         if d2 > CHI2_99[len(z)]:
33             continue                    # outlier
34
35         # Kalman gain
36         K = Sigma_bar @ H.T @ np.linalg.inv(S)
37
38         # Update
39         mu_bar = mu_bar + K @ innov
40         I_KH = np.eye(15) - K @ H
41         Sigma_bar = I_KH @ Sigma_bar @ I_KH.T + K @ Q @ K.T  # Joseph
42

```

```

43 mu, Sigma = mu_bar, Sigma_bar
44
45 # === PUBLISH ===
46 publish_tf("odom", "base_link", mu[:3], quat_from_euler(mu[3:6]))
47 publish_odometry("/odometry/filtered", mu, Sigma)

```

9.2 Multi-Sensor Update Order

When multiple measurements arrive in same cycle, process sequentially in time order. `robot_localization` buffers measurements by timestamp, processes chronologically. Allows *retroactive correction*: if delayed measurement arrives, filter rolled back to its timestamp, measurement applied, filter re-stepped forward. Implementation: ring buffer of past states.

9.3 Prediction-Only Periods

Between GPS fixes (e.g., 1 Hz GPS, 50 Hz filter):

```

t=0.00: GPS update    -> Sigma shrinks
t=0.02: Predict       -> Sigma grows slightly
...
t=0.98: Predict       -> Sigma grown considerably
t=1.00: GPS update    -> Sigma shrinks

```

“Breath” of filter. After 1 s of dead reckoning with $\sigma_{\dot{x}} = 0.1$ m/s, position uncertainty grows by ≈ 0.1 m.

10 Wheel Odometry: Theory and Practice

10.1 Hardware: Quadrature Encoders

A quadrature encoder is two photodetectors (or magnetic Hall sensors) reading a slotted disc on the wheel axle. Two channels A and B physically offset by 90° of the slot pattern, producing two square waves in quadrature:

```

Channel A: |##_|##_|##_|##_|
Channel B: ##|_||##_|##_|##_|

```

Forward rotation: A leads B. Reverse: B leads A. Counting transitions on both channels gives $4N$ counts per revolution where N is slot count — “ $\times 4$ decoding”. Typical robotics encoders: 1024–4096 CPR after $\times 4$. With gearing, output-shaft resolution much higher.

10.2 Differential-Drive Kinematics

Define:

- r : wheel radius
- b : wheel baseline (track width, distance between left/right wheels)
- $\Delta n_L, \Delta n_R$: encoder counts on left/right wheels during Δt
- N_{CPR} : counts per revolution (after $\times 4$)

Linear wheel velocities (m/s):

$$v_L = \frac{2\pi r \Delta n_L}{N_{\text{CPR}} \Delta t}, \quad v_R = \frac{2\pi r \Delta n_R}{N_{\text{CPR}} \Delta t}$$

Robot body-frame velocities:

$$\boxed{v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{b}} \quad (14)$$

These two scalars (v, ω) are the entire output of the kinematic model.

10.3 Pose Integration

Euler integration (first-order, accumulates more error):

$$x_{t+1} = x_t + v \cos(\theta_t) \Delta t, \quad y_{t+1} = y_t + v \sin(\theta_t) \Delta t, \quad \theta_{t+1} = \theta_t + \omega \Delta t$$

Runge-Kutta 2nd-order (midpoint) — evaluate orientation at midpoint:

$$x_{t+1} = x_t + v \cos\left(\theta_t + \frac{\omega \Delta t}{2}\right) \Delta t, \quad y_{t+1} = y_t + v \sin\left(\theta_t + \frac{\omega \Delta t}{2}\right) \Delta t$$

Exact integration for constant (v, ω) over interval (when $\omega \neq 0$, motion on circular arc):

$$\begin{aligned} \Delta x &= \frac{v}{\omega} [\sin(\theta_t + \omega \Delta t) - \sin(\theta_t)] \\ \Delta y &= -\frac{v}{\omega} [\cos(\theta_t + \omega \Delta t) - \cos(\theta_t)] \\ \Delta \theta &= \omega \Delta t \end{aligned}$$

For $\omega \rightarrow 0$: straight-line case recovered.

10.4 What the EKF Receives

Crucial choice: feed the EKF *integrated pose* or *velocity (twist)*?

Best practice: feed twist, not pose. EKF performs own integration, fusing wheel + IMU. Feeding pre-integrated pose causes double integration of encoder noise. Feed instantaneous velocities, let filter integrate with all sensors weighted properly.

```
1 odom0: /wheel_odom
2 odom0_config: [false, false, false,      # don't use position
3               false, false, false,      # don't use orientation
4               true, false, false,        # USE twist v_x
5               false, false, true,        # USE twist omega_z
6               false, false, false]      # don't use accel
```

10.5 Error Sources

1. Wheel slip. Actual ground travel differs from encoder count:

$$v_{\text{true}} = v_{\text{measured}} \cdot (1 - s), \quad s \in [0, 1]$$

s spikes near 1 on ice or sharp acceleration. EKF cannot distinguish slip from real motion — relies on IMU/GPS to catch.

2. Wheel radius miscalibration. Assumed $\hat{r}$ vs true r :

$$\frac{e_v}{v} = \frac{r - \hat{r}}{\hat{r}}$$

Constant scale error. Position drift grows linearly with distance. Tire wear and pressure variation drift this over time.

3. Baseline miscalibration. Assumed $\hat{b}$ vs true b :

$$\frac{e_\omega}{\omega} = \frac{b - \hat{b}}{\hat{b}}$$

Heading error per turn. Manifests as systematic curvature: robot commanded straight spirals slowly.

4. Encoder quantisation.

$$\sigma_{v,\text{quant}} \approx \frac{2\pi r}{N_{\text{CPR}} \Delta t \sqrt{12}}$$

5. Random walk drift. On flat terrain with i.i.d. noise, position uncertainty grows as $\sqrt{d}$ where d is distance travelled. Systematic bias makes error grow linearly.

10.6 Noise Model

$$Q_{\text{wheel}} = \begin{bmatrix} \sigma_{v_x}^2 & 0 \\ 0 & \sigma_{\omega_z}^2 \end{bmatrix}$$

Typical small mobile robot:

- $\sigma_{v_x} \approx 0.05\text{--}0.1$ m/s
- $\sigma_{\omega_z} \approx 0.02\text{--}0.05$ rad/s

Tune empirically: drive known trajectory, compare integrated pose to ground truth, increase covariance until residual consistent with gating.

11 IMU: Theory and Practice

11.1 MEMS Inertial Sensors

Modern IMUs are MEMS (Micro-Electro-Mechanical Systems) devices etched in silicon at micron scale.

11.1.1 Accelerometer

Proof mass m suspended by silicon spring beams between fixed electrodes. Linear acceleration along sensitive axis displaces mass, changing capacitance:

$$C = \frac{\varepsilon A}{d}$$

d is gap. Differential capacitive sensing measures gap change to nanometre precision. Deflection proportional to specific force along axis.

Critical fact: accelerometer measures *specific force*, not pure acceleration. Output:

$$\boxed{\tilde{a}^b = a_{\text{true}}^b - R_w^b g + b_a + n_a} \quad (15)$$

where R_w^b rotates world-frame gravity $g = [0, 0, -9.81]^\top$ into body frame. Free-falling accelerometer reads zero. Stationary accelerometer reads $-g$ rotated into body frame — pointing “up”.

11.1.2 Gyroscope (Coriolis MEMS)

Vibrating mass driven at resonant frequency along drive axis. Package rotation induces Coriolis force, deflecting mass along sense axis:

$$F_{\text{Coriolis}} = -2m(\Omega \times v_{\text{drive}})$$

Deflection amplitude proportional to angular velocity Ω . Output:

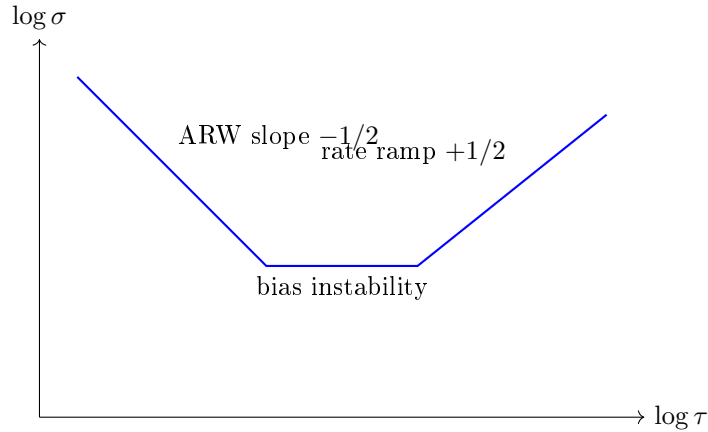
$$\tilde{\omega}^b = \omega_{\text{true}}^b + b_g + n_g \quad (16)$$

11.1.3 Magnetometer (9-DOF IMUs)

Hall-effect or magnetoresistive sensor measures local magnetic field. Ideally points to magnetic north, gives absolute yaw. Practice: corrupted by hard-iron (permanent magnets), soft-iron (ferromagnetic distortion), environmental anomalies (steel structures, motors). Indoor robots often disable magnetometer.

11.2 Allan Variance Noise Model

Allan variance plot is canonical IMU noise characterisation. Plot $\sigma_{\text{Allan}}(\tau)$ vs averaging time τ on log-log axes:



Parameters extracted:

- **Angle Random Walk (ARW)** [$^\circ/\sqrt{\text{hr}}$] for gyros: white-noise floor. Attitude error $\sigma_\theta(t) = \text{ARW}\sqrt{t}$.
- **Velocity Random Walk (VRW)** [$\text{m/s}/\sqrt{\text{hr}}$] for accelerometers. Velocity error $\sigma_v(t) = \text{VRW}\sqrt{t}$.
- **Bias Instability (BI)** [$^\circ/\text{hr}$] for gyros, [μg] for accels: minimum noise floor.

Full noise model:

$$\dot{b}_g(t) = n_{bg}(t), \quad n_{bg} \sim \mathcal{N}(0, \sigma_{bg}^2 I) \quad (17)$$

$$\dot{b}_a(t) = n_{ba}(t), \quad n_{ba} \sim \mathcal{N}(0, \sigma_{ba}^2 I) \quad (18)$$

Biases evolve as random walks — drift slowly but unboundedly.

11.3 Orientation Integration

Rotation matrix form:

$$\dot{R} = R[\omega]_{\times}$$

where $[\omega]_{\times}$ is skew-symmetric:

$$[\omega]_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

Quaternion form (preferred): Unit quaternion $q = [q_w, q_x, q_y, q_z]^{\top}$, $\|q\| = 1$:

$$\dot{q} = \frac{1}{2} q \otimes \omega_q, \quad \omega_q = [0, \omega_x, \omega_y, \omega_z]^{\top}$$

Discrete update over Δt :

$$q_{t+\Delta t} = q_t \otimes \exp\left(\frac{1}{2} \omega_q \Delta t\right)$$

followed by normalisation.

11.4 Attitude from Accelerometer (Static Case)

Stationary or constant velocity: $a_{\text{true}} \approx 0$ and:

$$\tilde{a}^b \approx -R_w^b g$$

Accelerometer measures gravity vector in body frame. Recover roll ϕ and pitch θ (not yaw):

$$\phi = \text{atan2}(\tilde{a}_y, \tilde{a}_z), \quad \theta = \text{atan2}(-\tilde{a}_x, \sqrt{\tilde{a}_y^2 + \tilde{a}_z^2})$$

Absolute roll/pitch reference — no drift! Why IMU is so valuable: bounds roll/pitch error indefinitely (modulo bias). Yaw drifts unboundedly without magnetometer or external reference.

11.5 Madgwick Complementary Filter

For attitude estimate from IMU alone, the Madgwick filter (used in `imu_filter_madgwick`) combines gyro integration with gradient-descent correction toward gravity (and magnetic field if available):

$$q_{t+1} = q_t + \Delta t (\dot{q}_{\omega,t} - \beta \nabla F(q_t))$$

where ∇F is the gradient of an objective measuring agreement between q_t 's predicted gravity direction and the measured accelerometer vector. β is the gain. Cheap, robust, runs on microcontrollers.

11.6 What the EKF Receives

```

1 imu0: /imu/data
2 imu0_config: [false, false, false,      # no position
3               true,  true,  false,      # USE roll, pitch (no yaw if no mag)
4               false, false, false,      # no linear velocity
5               true,  true,  true,        # USE angular velocity
6               true,  true,  false]      # USE planar accel (z is gravity -
           contaminated)
7 imu0_remove_gravitational_acceleration: true

```

Roll and pitch reliable from gravity vector. Yaw from magnetometer often noisy — prefer gyro yaw, rely on visual/LiDAR/GPS heading. Angular velocity is rate signal, EKF integrates. Linear acceleration tricky: includes gravity unless subtracted via $R_w^b g$. Vertical channel often disabled.

12 Visual Odometry: Theory and Practice

12.1 What VO Computes

Camera ego-motion estimation between frames. Outputs chain of relative transformations $\Delta T_1, \Delta T_2, \dots$ composed into trajectory:

$$T_{\text{world}}^{\text{cam}}(t) = T_{\text{world}}^{\text{cam}}(0) \cdot \Delta T_1 \cdots \Delta T_t$$

12.2 Camera Model

Pinhole camera projection:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[R \mid t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

- (X, Y, Z) : 3D point in world coordinates
- (u, v) : pixel coordinates
- K : intrinsic matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- $[R \mid t]$: extrinsic matrix (camera pose)
- λ : depth scaling factor

Lens distortion (radial + tangential, Brown-Conrady) corrected before projection.

12.3 Feature-Based VO Pipeline

12.3.1 Step 1: Feature Detection

Find repeatable, distinctive, real-time keypoints.

FAST: For each pixel p , examine 16 pixels on Bresenham circle of radius 3. If ≥ 9 contiguous pixels brighter than $I(p) + t$ or darker than $I(p) - t$, mark p as corner. Extremely fast.

ORB (Oriented FAST + Rotated BRIEF): add orientation via intensity centroid:

$$\theta = \text{atan2}(m_{01}, m_{10}), \quad m_{pq} = \sum_{(x,y) \in \text{patch}} x^p y^q I(x, y)$$

12.3.2 Step 2: Feature Description

Descriptor = patch fingerprint. ORB uses BRIEF: compare intensities at 256 random pixel pairs $\rightarrow$ 256-bit binary string. Match by Hamming distance (XOR + popcount) — extremely fast.

12.3.3 Step 3: Feature Matching

For each keypoint in frame t , find closest descriptor in frame $t - 1$. Apply Lowe's ratio test:

$$\frac{d_{\text{best}}}{d_{\text{second}}} < 0.7$$

Keep match only if best is significantly better than second-best.

12.3.4 Step 4: RANSAC for Outlier Rejection

Wrong matches inevitable. RANSAC robustly fits model in presence of outliers:

```

1 best_inliers = []
2 for iter in range(N):
3     sample = random_minimal_subset(matches)
4     hypothesis = compute_motion(sample)
5     inliers = [m for m in matches if reprojection_error(m, hypothesis) < tau]
6     if len(inliers) > len(best_inliers):
7         best_inliers, best_hypothesis = inliers, hypothesis
8 final_motion = refine(best_hypothesis, best_inliers)

```

Iterations N for probability p of finding all-inlier sample with sample size s and outlier ratio ε :

$$N = \frac{\log(1-p)}{\log(1-(1-\varepsilon)^s)}$$

12.3.5 Step 5: Motion Recovery

Monocular: Essential Matrix. Normalised correspondences $x_1 \leftrightarrow x_2$:

$$x_2^\top E x_1 = 0, \quad E = [t]_\times R$$

≥ 5 correspondences recover E (Nistér's 5-point). SVD of E yields R, t but t recovered only up to scale. Monocular VO has *scale ambiguity*.

Stereo: Triangulation + PnP. Known baseline b :

1. Disparity d : $d = u_L - u_R$
2. Depth: $Z = f \cdot b/d$
3. 3D point: $(X, Y, Z) = ((u - c_x)Z/f, (v - c_y)Z/f, Z)$
4. PnP: given 3D-to-2D correspondences in next frame, solve for new pose.

PnP via DLT. For 3D point X and 2D measurement m :

$$m \times (PX) = 0, \quad P = K[R|t]$$

$n \geq 6$ correspondences $\rightarrow$ homogeneous linear system $Ap = 0$, solved by SVD.

PnP refinement. Minimise reprojection error:

$$\min_{R,t} \sum_i \|\pi(K, R, t, X_i) - m_i\|^2$$

via Levenberg-Marquardt.

12.4 Direct VO Methods

Skip features. Minimise photometric error:

$$E(\xi) = \sum_{p \in \Omega} \rho(I_2(\pi(\xi, p, d_p)) - I_1(p))$$

$\xi \in \mathfrak{se}(3)$ camera motion, d_p depth at pixel, ρ robust loss. Optimised by Gauss-Newton.

DSO, LSD-SLAM, DTAM. Works in low-texture (gradient information dense). Fails under lighting changes.

12.5 Visual-Inertial Odometry (VIO)

Tight coupling of camera + IMU. IMU preintegrated between camera frames:

$$\Delta R_{ij} = \prod_{k=i}^{j-1} \exp((\tilde{\omega}_k - b_g)\Delta t) \quad (19)$$

$$\Delta v_{ij} = \sum_{k=i}^{j-1} R_{ik}(\tilde{a}_k - b_a)\Delta t \quad (20)$$

$$\Delta p_{ij} = \sum_{k=i}^{j-1} [\Delta v_{ik}\Delta t + \frac{1}{2}R_{ik}(\tilde{a}_k - b_a)\Delta t^2] \quad (21)$$

Preintegrated quantities become factors in graph optimisation alongside visual reprojection factors.

VINS-Mono, OpenVINS, ORB-SLAM3-VI, Kimera-VIO.

12.6 VO Output for the EKF

nav_msgs/Odometry:

- **pose.pose**: 6-DOF camera (or robot via extrinsic) pose
- **pose.covariance**: 6×6 uncertainty
- **twist.twist**: linear/angular velocity in body frame
- **twist.covariance**: 6×6

Covariance estimated from inverse observed information matrix at optimum (Cramér-Rao bound), with empirical scaling.

12.7 Failure Modes

- **Low-texture**: corridors, white walls — few features, ambiguous matching
- **Motion blur**: fast rotation smears features
- **Lighting changes**: shadows, glare, auto-exposure shifts break photometric consistency
- **Dynamic objects**: people, cars bias static-scene assumption
- **Rolling shutter**: each row captured at different time — distorts geometry under motion
- **Scale drift (monocular)**: no metric reference; tightly coupled IMU mostly fixes

13 LiDAR Odometry: Theory and Practice

13.1 Time-of-Flight Range Measurement

LiDAR fires laser pulses (905 nm or 1550 nm), measures round-trip:

$$d = \frac{c \cdot \Delta t}{2}, \quad c = 3 \times 10^8 \text{ m/s}$$

At 100 m range, round trip ~ 667 ns. Sub-ns timing $\rightarrow$ sub-15 cm precision; better via analog waveform fitting.

Rotating LiDAR (Velodyne, Ouster): 16–128 vertical beams rotated through 360° at 5–20 Hz. Each revolution = one scan, 100k–2M points.

Solid-state LiDAR (Livox, Innoviz): no moving parts, MEMS or optical-phased-array beam steering. Limited FOV, higher density.

Each point: $(x, y, z, \text{intensity}, \text{ring}, \text{timestamp})$ in sensor frame.

13.2 Iterative Closest Point (ICP)

Canonical algorithm for point cloud alignment. Source S , target T . Find rigid transform (R, t) minimising:

$$E(R, t) = \sum_{i=1}^N \|t_{c(i)} - (Rs_i + t)\|^2$$

$c(i)$ = index of target point matched to source i .

13.2.1 Algorithm

```

1 R, t = R_init, t_init
2 while not converged:
3     S_curr = [R @ s + t for s in S]
4     correspondences = [(i, nearest_idx(S_curr[i], T)) for i in range(len(S))]
5     correspondences = filter_outliers(correspondences, max_dist)
6
7     s_bar = mean([S[i] for (i,j) in correspondences])
8     t_bar = mean([T[j] for (i,j) in correspondences])
9     H_mat = sum((S[i] - s_bar) @ (T[j] - t_bar).T for (i,j) in correspondences)
10    U, Sigma_, Vt = svd(H_mat)
11    R_new = Vt.T @ U.T
12    if det(R_new) < 0:
13        Vt[-1,:] *= -1
14        R_new = Vt.T @ U.T
15    t_new = t_bar - R_new @ s_bar
16    R, t = R_new @ R, R_new @ t + t_new

```

13.2.2 Why SVD Yields Optimal Rotation

Given correspondences, minimise $\sum_i \|t_i - (Rs_i + t)\|^2$. Centering eliminates t :

$$\min_R \sum_i \|\tilde{t}_i - R\tilde{s}_i\|^2 = \min_R \sum_i \|\tilde{t}_i\|^2 + \|\tilde{s}_i\|^2 - 2\tilde{t}_i^\top R\tilde{s}_i$$

Reduces to $\max_R \text{tr}(RH^\top)$ where $H = \sum_i \tilde{s}_i \tilde{t}_i^\top$. Procrustes theorem: $R^* = VU^\top$ where $H = U\Sigma V^\top$.

13.3 Point-to-Plane ICP

Better convergence on surface-rich data. Minimise distance from source point to plane through target with normal n_i :

$$E = \sum_i (n_i^\top (Rs_i + t - t_i))^2$$

Normals concentrate error along physically meaningful surface directions. Sliding along wall = zero error. Convergence dramatically faster on structured environments.

13.4 Normal Distributions Transform (NDT)

Faster for sparse clouds. Voxelise target; for each voxel with $\geq k$ points, fit Gaussian $\mathcal{N}(\mu_k, \Sigma_k)$. Transformed source point p' , score:

$$s(p') = \exp\left(-\frac{1}{2}(p' - \mu_k)^\top \Sigma_k^{-1}(p' - \mu_k)\right)$$

Maximise $\sum s(p'_i)$ over (R, t) via Newton's method. Differentiable, avoids nearest-neighbour search per iteration.

13.5 LOAM and LIO-SAM

LOAM two-stage design:

1. Segment scan into *edge* (high curvature) + *planar* (low curvature) points.
2. High-frequency odometry (10 Hz): match edges/planes against previous scan.
3. Low-frequency mapping (1 Hz): refine pose against accumulated point cloud map.

LIO-SAM adds tightly coupled IMU + factor graph. Factors:

- IMU preintegration between scans (motion prior)
- Scan-matching constraints (LOAM-style edge/plane factors)
- Loop closure factors (revisits)
- Optional GPS factors

Graph optimised by iSAM2 (incremental smoothing and mapping). Globally consistent trajectory.

13.6 LiDAR Output for the EKF

nav_msgs/Odometry or TransformStamped:

- Pose: 6-DOF in odom or local fixed frame
- Pose covariance: estimated from information matrix at ICP convergence
- Twist: optional linear/angular velocity

In structured environment, exceptionally accurate: $\sigma_x \approx 0.01$ m, $\sigma_\theta \approx 10^{-4}$ rad. In featureless terrain, degenerates — inflate Q_t .

13.7 Failure Modes

- **Geometric degeneracy:** long corridor, open field — ambiguity along DOFs
- **Dynamic objects:** bias registration
- **Weather:** rain, fog, snow, dust scatter laser
- **Scan distortion:** rotating LiDARs sample ~ 100 ms while moving; without IMU motion correction, scan distorted
- **Aliasing:** repetitive structures (fences, columns) admit multiple consistent registrations

14 GPS / GNSS

14.1 Principle

GNSS satellites broadcast precisely timed signals. Receiver measures time-of-flight from ≥ 4 satellites, triangulates 3D position.

Pseudorange from satellite i :

$$\rho_i = c(t_{\text{rx}} - t_{\text{tx}}^i) = \|p_{\text{rx}} - p^i\| + c\delta t_{\text{rx}} + \varepsilon_i$$

δt_{rx} = unknown receiver clock bias. Four equations, four unknowns $(x, y, z, \delta t)$, solved by Newton's method on linearised system.

14.2 Accuracy Tiers

Type	Accuracy	Notes
Standard GPS	3–10 m horizontal	Atmospheric errors dominate
SBAS (WAAS/EGNOS)	1–3 m	Geostationary correction signal
DGPS	< 1 m	Local base station corrections
RTK	1–3 cm horizontal	Carrier-phase resolution, base required
PPK	sub-cm	Offline post-processing

14.3 What GPS Provides

- **Absolute position** (lat, lon, alt) — no drift, but noisy moment-to-moment
- **No heading** from single antenna. Velocity heading from Doppler available only when moving
- **Dual-antenna RTK**: absolute heading from triangulating antenna baseline

14.4 GPS in the Filter

Canonical pattern: consumed by global EKF only — never local EKF (jumps would corrupt smooth local estimate). `navsat_transform_node` converts (lat, lon) to map-frame (x, y); global EKF treats as position measurement with covariance from receiver's HDOP.

Project pattern (this codebase): `navsat_transform_node` still runs and publishes `/odometry/gps`, but neither EKF subscribes to it. GPS available for analysis notebooks (e.g., compare filtered estimate against GPS truth offline) but does *not* influence the filter. Reason: the covariance cross-term leak (Section 8.5) caused $3.56^\circ/\text{s}$ yaw drift when GPS was fused; without a strong absolute yaw observation to fight the leak, sensor consensus across both EKFs (identical inputs) is preferred over GPS anchoring.

Bottom line: GPS still flows through the pipeline, but is excluded from EKF inputs. Map frame remains stable at the cost of global anchoring.

15 Tuning the Filter

15.1 Process Noise R_t

R_t models how state can change between predict steps beyond motion model. Filter's "humility" — how much it doesn't trust its own dynamics.

Effect: High $R_t \rightarrow$ trust measurements more, respond faster, noisier. Low $R_t \rightarrow$ smoother, slower to respond.

Typical diagonal for small ground robot (per-second, internally scaled by Δt):

- Position: 0.05–0.1 m²/s
- Orientation: 0.01–0.05 rad²/s
- Velocity: 0.05–0.2 (m/s)²/s
- Angular velocity: 0.01–0.05 (rad/s)²/s
- Acceleration: 0.1–0.5 (m/s²)²/s

15.2 Measurement Noise Q_t

Trust in each sensor. Sources:

1. Datasheet specs (gyro ARW, GPS HDOP)
2. Empirical: static or known-trajectory data, residual variance
3. Allan variance for IMU
4. Reported covariance from upstream nodes

15.3 Initial Covariance Σ_0

Starting uncertainty:

- Position: 1–10 m² (often known at origin, smaller)
- Orientation: 1 rad² for unknown heading
- Velocity: 0.5 (m/s)²

15.4 Validation: Innovation Whiteness

Innovation sequence $z_t - h(\bar{\mu}_t)$ should be:

- **Zero mean:** no bias
- **White:** no temporal correlation
- **Consistent:** variance matches $S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t$

Normalised Innovation Squared (NIS):

$$\text{NIS}_t = (z_t - h(\bar{\mu}_t))^\top S_t^{-1} (z_t - h(\bar{\mu}_t))$$

Should follow χ_k^2 with $k =$ measurement dimension. Consistently large NIS $\rightarrow$ overconfident (small R_t or Q_t). Consistently small $\rightarrow$ underconfident.

15.5 Common Failure Modes

Filter Divergence Causes

1. Linearisation error grows beyond what R_t absorbs (highly nonlinear motion)
2. Forgotten angle wrapping in innovation
3. Σ loses positive-definiteness (use Joseph form)
4. Outlier measurements not gated
5. Sensor timestamps misaligned (one sensor lagging)
6. Sensor frames misconfigured (TF errors)
7. Double-counting same information through multiple paths

16 Beyond the EKF

16.1 Unscented Kalman Filter (UKF)

EKF linearises by Taylor expansion — first-order. Highly nonlinear systems: fails. UKF propagates deterministic set of *sigma points*:

1. Choose $2n + 1$ sigma points around μ :

$$\begin{aligned}\chi_0 &= \mu \\ \chi_i &= \mu + (\sqrt{(n + \lambda)\Sigma})_i, \quad i = 1, \dots, n \\ \chi_{n+i} &= \mu - (\sqrt{(n + \lambda)\Sigma})_i, \quad i = 1, \dots, n\end{aligned}$$

2. Propagate through nonlinear function: $\chi'_i = g(\chi_i)$
3. Reconstruct mean and covariance from weighted sigma points.

No Jacobian. Captures nonlinearity to second order. Slightly more expensive but more accurate for strongly nonlinear systems.

16.2 Particle Filter

Belief as swarm of weighted samples. Multimodal, non-Gaussian distributions — crucial for global localisation (kidnapped robot). Used in ROS via AMCL.

Cost $O(N)$ per step. N must be large in high dimensions (curse of dimensionality).

16.3 Factor Graphs and iSAM

Modern SLAM systems (GTSAM, Ceres) optimise entire trajectory simultaneously. Each measurement = factor in probabilistic graph. iSAM2 makes this real-time by reusing previous solution. More accurate than EKF over long trajectories with loop closures; more expensive.

17 Putting It All Together

17.1 End-to-End Data Flow (this Project)

1. Wheel encoders count pulses. `diff_drive_controller` computes (v, ω) via diff-drive kinematics. Publishes `/diff_drive_controller/odom` at ~ 50 Hz.
2. IMU samples accel + gyro. `imu_filter_madgwick` produces orientation. Publishes `/imu/data` at ~ 100 – 200 Hz.

3. LiDAR rotates, samples points. RTAB-Map `icp_odometry` performs scan matching; publishes `/odometry/lidar` at ~ 10 Hz.
4. RealSense D435 captures RGB + depth. RTAB-Map `rgbd_odometry` detects GFTT features, matches Frame-to-Map, runs PnP solver; publishes `/odometry/visual` at ~ 14.3 Hz.
5. Gazebo NavSat plugin publishes `/gps/fix` at 10 Hz with $\sigma_{\text{horiz}} \approx 0.5$ m noise.
6. **Local EKF** (`world_frame: odom`): predicts at ~ 50 Hz. Updates with each wheel/IMU/LiDAR/VO measurement. Publishes `/odometry/filtered` (smooth). Publishes `odom` $\rightarrow$ `base_link` TF.
7. `navsat_transform_node`: receives GPS + IMU heading + `/odometry/filtered`. On first fix, establishes datum (UTM Zone 32U for Gazebo origin at 49.9°N, 8.9°E). Publishes `/odometry/gps` for analysis. *Not consumed by either EKF in this project.*
8. **Global EKF** (`world_frame: map`): subscribes to the *same raw sensors* as local EKF (wheel + IMU + LiDAR + VO), **not** `/odometry/gps`. Predicts/updates at ~ 50 Hz. Publishes `/odometry/filtered_map` and `map` $\rightarrow$ `odom` TF (near-identity).
9. Navigation stack consumes both transforms: global path planner uses `map`, local controller uses `odom`. Map frame anchored at robot's spawn position (sensor consensus, not GPS).

17.2 Why Each Component Exists

- **Wheel odometry**: cheap, high-rate baseline. Anchors short-term motion.
- **IMU**: bounds roll/pitch absolutely (gravity); high-rate angular velocity.
- **LiDAR/Visual odometry**: bounds drift in x, y, θ via scan-matching/feature-tracking.
- **GPS** (project): *not fused into EKFs* due to covariance leak. Available via `/odometry/gps` for analysis; would provide global anchoring if a strong yaw observation existed to suppress the leak.
- **Local EKF**: smooth, continuous estimate for control. Drifts in absolute frame.
- **Global EKF**: in canonical pattern, GPS-corrected globally consistent estimate. In this project, redundant filter with `world_frame: map` that produces near-identical output to local EKF, anchoring map frame at spawn position.

Each sensor compensates for failure modes of others. Wheel encoders fail on slip — IMU detects unexpected motion. IMU drifts in yaw — LiDAR/VO bound it. LiDAR fails in long corridors — wheel/IMU/VO keep going. Redundancy = entire point of architecture.

17.3 Comparison: Canonical vs Project Pattern

Aspect	Canonical (Tom Moore)	Project (Husarion Lynx)
Local EKF inputs	wheel + IMU	wheel + IMU + VO + LiDAR
Global EKF inputs	wheel + IMU + GPS	wheel + IMU + VO + LiDAR (no GPS)
GPS role	fused into global EKF	published only, not fused
<code>map</code> $\rightarrow$ <code>odom</code>	jumps on GPS correction	near-identity (sensor consensus)
Yaw stability	relies on dual-antenna RTK or magnetometer	sensor consensus suppresses leak
Global anchoring	yes (GPS)	no (drifts with odom)
Best for	open outdoor + RTK	indoor / GPS-denied / short-range outdoor
Failure mode if mismatched	yaw cross-term leak ($\sim 3.56^\circ/\text{s}$)	no global georeference

18 Summary Cheat Sheet

18.1 The KF in One Page

Algorithm: Kalman_filter

Inputs: $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$

Prediction step:

$$\begin{aligned}\bar{\mu}_t &= A_t \mu_{t-1} + B_t u_t \\ \bar{\Sigma}_t &= A_t \Sigma_{t-1} A_t^\top + R_t\end{aligned}$$

Correction step:

$$\begin{aligned}K_t &= \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1} \\ \mu_t &= \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ \Sigma_t &= (I - K_t C_t) \bar{\Sigma}_t\end{aligned}$$

Return μ_t, Σ_t

18.2 The EKF in One Page

Algorithm: Extended_Kalman_filter

Inputs: $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$

Compute Jacobians:

$$G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}, \quad H_t = \frac{\partial h(\bar{\mu}_t)}{\partial x_t}$$

Prediction step:

$$\begin{aligned}\bar{\mu}_t &= g(u_t, \mu_{t-1}) \\ \bar{\Sigma}_t &= G_t \Sigma_{t-1} G_t^\top + R_t\end{aligned}$$

Correction step:

$$\begin{aligned}K_t &= \bar{\Sigma}_t H_t^\top (H_t \bar{\Sigma}_t H_t^\top + Q_t)^{-1} \\ \mu_t &= \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t)) \\ \Sigma_t &= (I - K_t H_t) \bar{\Sigma}_t\end{aligned}$$

Return μ_t, Σ_t

18.3 Sensor Quick Reference

Sensor	Rate	Provides	Bounds	Drifts on
Wheel encoder	20–50 Hz	v_x, ω_z	—	slip, r/b error
IMU 6-DOF	100–200 Hz	ang vel, accel, roll/pitch	ϕ, θ	yaw
IMU 9-DOF	100–200 Hz	+ magnetometer yaw	ϕ, θ, ψ	magnetic anomalies
Visual odom	10–30 Hz	$\Delta T_{6\text{DOF}}$	—	low texture, scale
LiDAR odom	5–20 Hz	$\Delta T_{6\text{DOF}}$	—	corridors, weather
GPS	1–10 Hz	abs. position	global drift	indoor, multipath
RTK dual-ant.	5–10 Hz	abs. pos. + heading	global, ψ	indoor

18.4 Frame Hierarchy Quick Reference

Transform	Source	Property
map $\rightarrow$ odom	global EKF	can jump (GPS corrections)
odom $\rightarrow$ base_link	local EKF	continuous (no jumps)
base_link $\rightarrow$ sensor	URDF/static	rigid

18.5 Topic / Node Quick Reference

Component	Inputs	Outputs
ekf_node_local	wheel, IMU, LiDAR/VO	/odometry/filtered, TF odom $\rightarrow$ base_link
navsat_transform_node	GPS, IMU, /odometry/filtered	/odometry/gps
ekf_node_global	/odometry/filtered, /odometry/gps, IMU	/odometry/filtered_map, TF map $\rightarrow$ odom
imu_filter_madgwick	/imu/data_raw	/imu/data

18.6 Notation Cheat Sheet

Symbol	Meaning	Note
μ_t	state mean	
Σ_t	state covariance	
$\bar{\mu}_t, \bar{\Sigma}_t$	predicted mean/cov	before correction
$g(u_t, \mu_{t-1})$	motion model	nonlinear
$h(\bar{\mu}_t)$	observation model	nonlinear
G_t	motion Jacobian	$\partial g / \partial x$
H_t	observation Jacobian	$\partial h / \partial x$
A_t, C_t	linear motion / obs matrices	KF only
R_t	process noise covariance	motion uncertainty
Q_t	measurement noise covariance	sensor uncertainty
K_t	Kalman gain	
u_t, z_t	control, measurement	

Closing Notes

The Extended Kalman Filter is not a single algorithm but a principle: maintain a Gaussian belief over the state, propagate it through your best model of the world, and correct it with every measurement weighted by its uncertainty. Every line of the algorithm follows from this principle plus the linearisation that makes it tractable.

The dual-EKF architecture used in `robot_localization` is a direct consequence of the constraint that the controller’s pose source must be smooth, while the planner’s pose source must

be globally consistent. These goals are incompatible in a single filter. Two filters, one absorbing the other's drift via the `map` $\rightarrow$ `odom` transform, resolves the contradiction cleanly. The `navsat_transform_node` bridges raw GPS to the map-frame coordinate system, allowing GPS to enter the global filter without contaminating the local filter.

The sensor models in this document — wheel kinematics, IMU physics, visual feature geometry, LiDAR scan matching — are the languages in which each sensor speaks to the filter. The covariances Q_t are the volume controls on those voices. The process noise R_t is the filter's admission of its own ignorance about the world's dynamics. Tuning all of these together is the engineering practice; the math is just the framework that makes the tuning meaningful.